Quest 13: The Tickursion

Lord of the Wings · counting when the hands of a three-handed clock meet

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. It explains what the quest asks and the logic behind my solutions, not the code itself. Every number in the examples and traces was checked by running my solutions on the sample data (plus small instrumented copies to print intermediate values).

THE QUESTION

A clock has three hands, each with its own number of positions, direction, start position and number of seconds left to move. All move at once, smoothly, one position per second, until each runs out. Count how many times a pair of hands comes to point the same way.

INPUT

Four lines per clock: `hands`, `direction`, `current`, `move`, values for hands 1:2:3. Parts II and III list many clocks separated by blank lines.

OUTPUT

The number of meetings (signals) for each clock; in Parts II and III, their sum.

WHAT CHANGES BETWEEN PARTS

Part I: one clock. Part II: many clocks, moves in the thousands. Part III: same, but moves up to about 10^{12} seconds, so stepping through every second is out.

The story

Having beaten Stackrog for good, the narrator becomes Ganduck the Whitelog and can now see the portals all around the Abyss of Unmarked Recursion. Each portal is opened by setting a dial to the number of signals its three-handed clock will still send. One portal leads to the next, chamber after chamber, and the later clocks keep running for absurdly long.

Understanding the question

The rules

- A hand with *k* positions has them numbered 0 … *k* – 1 clockwise, equally spaced. Position 0 points the same way for every hand.
- Each hand moves one of **its own** positions per second (so a 6-position hand turns faster than a 12-position one), smoothly, so two hands can meet **part-way through** a second. When its moves run out it stops for good; `0` means already stopped.
- A **meeting** = a pair pointing the same way after pointing different ways. Each pair counts separately, so all three arriving together is 3 signals. Pointing the same way at the very start is **not** a meeting; meeting exactly at the end **is**.
- A pair that stays together (both stopped, or moving in step) doesn't keep meeting: they must separate first.

The official examples (all with hands 6:8:12, directions CW:CCW:CW)

	Example A	Example B	Example C
`current`	2:4:8	1:2:2	1:2:2
`move`	9:7:4	10:12:15	10:12:20
Meetings	1 + 3 + 1	seven single meetings	B's seven + two more
Signals	**5**	**7**	**9**

In B and C, hands 1 and 3 start overlapping (that doesn't count). In C they also finish together (that does). C is `sample1.txt`.

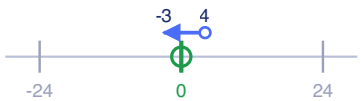
Extra examples of my own (run through copies of my Part I and Part III solutions)

Clock (hands / direction / current / move)	Signals	Why
4:4 / CW:CW / 0:0 / 8:0	2	Hand 1 laps the stopped hand 2 twice, after 4 and 8 seconds. The start doesn't count.
2:4 / CW:CCW / 0:1 / 1:1	1	They meet a third of the way through the only second, between two positions.
12:12 / CW:CW / 3:3 / 5:5	0	Same spot, same speed: they never separate, so they never meet.
6:4 / CW:CW / 0:0 / 6:4	1	Both do one full lap from 0. Hand 2 is faster, stops back at 0 after 4 s, and hand 1 arrives there 2 s later.

Shared approach: a common ruler and the gap trick

Common ruler. Positions of different hands aren't comparable directly, so all three are measured in the same small unit: L = the least common multiple of the position counts. For 6:8:12, L = **24**, and one step is 4, 3 and 2 units. Example A's start 2:4:8 becomes 8:12:16 units: hands 1 and 2 are 4 units (60°) apart.

Gap trick. For each pair, follow the **gap** = how far hand *b* is clockwise ahead of hand *a*, starting in 0 … L – 1 and **not** wrapped afterwards. While neither hand changes speed, the gap moves in a straight line, and the hands point the same way exactly when the gap is a multiple of L. So the number of meetings is just the number of multiples of L the gap **reaches**, leaving out where it starts (already together doesn't count) but including where it ends:



A, second 1, hands 1–2:
4 → –3 reaches 0: **+1**



B, second 11, hands 2–3:
starts on 0 (together): **+0**



C, second 20, hands 1–3:
ends exactly on 24: **+1**

Values from an instrumented copy of my Part I solution. Blue arrow = how the gap changes; green = the multiples of 24 it reaches, each one a meeting.

PART I One clock

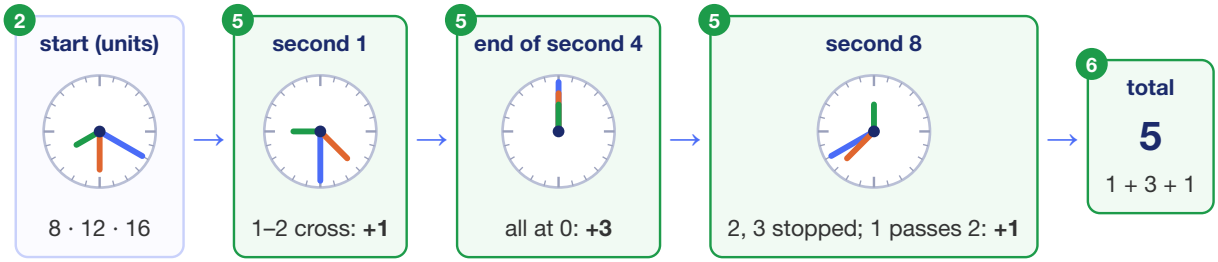
Given one clock's notes, how many signals will it send before all three hands stop?

Step-by-step solution

1. Read the clock: positions, direction, start position and moves left for each hand.
2. Put all hands on a common ruler: L = least common multiple of the position counts, each hand's step = L ÷ its count, and scale the start positions.
3. Simulate second by second up to the longest move. A hand with moves left moves +step (CW) or −step (CCW), otherwise 0. Record each second's start positions and moves.
4. For each second and each pair: take the gap from hand a to hand b at the start of the second (0 … L − 1), and how much it changes in that second.
5. Count the multiples of L the gap reaches, start excluded, end included, and add them to the total.
6. Print the total.

How my solution works

Put every hand on the common ruler, then **simulate one second at a time**, recording where each hand starts and how far it moves. Within a single second every hand has a constant speed, so for each second and each pair the gap trick counts that second's meetings exactly, including ones part-way through the second. Example A:



Dots match the step numbers above. **hand 1 (blue, longest)** **hand 2 (orange)** **hand 3 (green, shortest)**. Positions are in ruler units (24 per turn) from an instrumented copy. At second 4 every pair's gap lands on a multiple of 24 at once, so each pair counts 1.

On `sample1.txt` (example C) it prints **9**. The pair loop works for any number of hands, not just three.

PART II Many clocks

Same clocks, but now a whole list of them, some running for thousands of seconds. Add up every clock's signal count.

Worked example

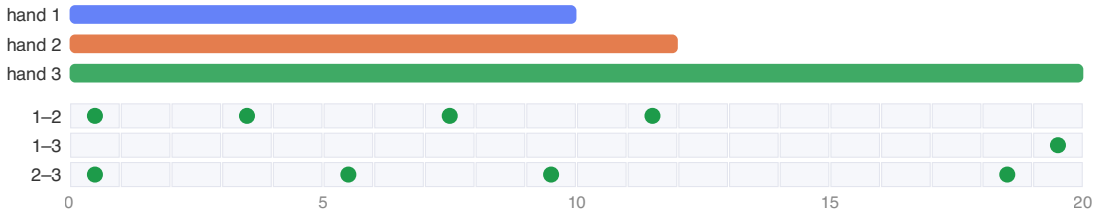
`sample2.txt` is examples A, B and C: 5 + 7 + 9 = **21**, which my Part II solution prints.

Step-by-step solution

1. Split the notes into clocks at the blank lines.
2. For each clock, count its signals exactly as in Part I.
3. Add up the counts and print the sum.

How my solution works

Nothing about one clock changes: Part I's counting is wrapped up as "signals for one clock", run on each block of notes, and the results are added. Every meeting in example C, second by second:



From an instrumented copy. Bars: the seconds each hand is still moving. Green dots: the second in which each pair meets (9 in all). Hands 1 and 3 meet only at the very end, after hands 1 and 2 have stopped.

> **Cost:** one step per second per clock, plus one record of all hand positions per second, so time and memory grow with the longest move. Fine for thousands of seconds (the real input runs in about 0.1 s); hopeless for 10^{12}.

PART III Enormous move counts

The same clocks again, but some hands move for around a trillion seconds. Find the sum of signals without simulating every second.

The idea: only stops change anything

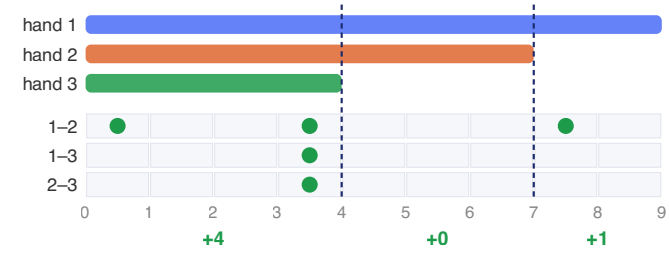
The gap trick never needed one-second steps, only a **constant speed** for every hand. Speeds change only when a hand stops, so cut time into at most three **stretches** between stops and count each one in one go: the work no longer depends on how long the hands move. `sample3.txt` holds examples A, B and C again (the puzzle's bigger example isn't saved); my solution prints **21**.

Step-by-step solution

1. Split the notes into clocks and put each clock on its common ruler, as before.
2. Make a sorted list of times: 0 and each hand's move count. Consecutive times bound the stretches; skip empty ones.
3. In each stretch, a hand still moving travels ±step × stretch length; a stopped hand travels 0.
4. For each pair, apply the gap count to the whole stretch at once.
5. Move every hand to its end position and go on to the next stretch. ↻ repeat
6. Add up the signals of all clocks and print the sum.

How my solution works

For each stretch between stop times, moving hands travel step × length, the gap trick counts each pair's meetings for the whole stretch, and the hands jump to their end positions. Example A:



4 **stretch 0–4 s**
start 8 · 12 · 16
moves +16 · –12 · +8
1–2: **+2** · 1–3: +1 · 2–3: +1

→

4 **stretch 4–7 s**
start 0 · 0 · 0
moves +12 · –9 · 0
all start together: +0

→

4 **stretch 7–9 s**
start 12 · 15 · 0
moves +8 · 0 · 0
1–2: **+1**

→

6 **total**
5
4 + 0 + 1

Dots match the step numbers above; values from an instrumented copy (ruler units). Dashed lines: stops. The first stretch's 1–2 gap falls from 4 to –24, reaching 0 and –24: seconds 1 and 4 in one go. Example C: 6 + 1 + 2 = 9.

Fragile: large numbers. Positions are ordinary floating-point numbers, exact only up to about 9×10^{15}. Ruler size × move count gets fairly close: on my input it peaks near 6% of that limit, so the answer is exact, but moves a few times bigger would miscount silently. Big integers would remove the limit.

Summary

Part	Clocks	Moves	Time is cut into	Work per clock	Sample answer
I	1	tens	single seconds	longest move × 3 pairs	9
II	many	thousands	single seconds	longest move × 3 pairs	21
III	many	up to ~10^{12}	≤ 3 stretches between stops	≤ 3 × 3 pairs	21

Key ideas: put everything on a **common ruler** (least common multiple) so positions compare as whole numbers. Turn "do two hands point the same way?" into "**is their gap a multiple of L?**", then count multiples crossed in a range with floor and ceiling division, with the start excluded and the end included. Since only stops change speeds, count whole **constant-speed stretches** at once instead of simulating each second.